

Portal Shadow SPA Framework

Версия: demo / experimental

Назначение: модульный SPA-портал с ленивой загрузкой контента и микросервисных компонентов

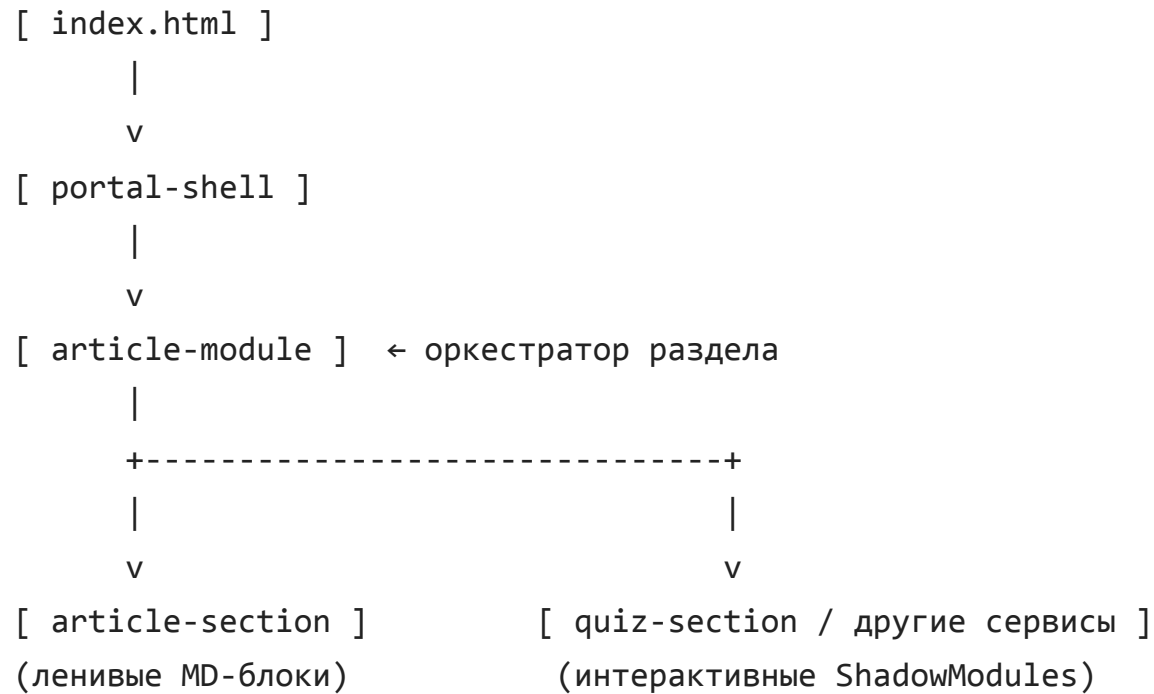
1. Идеология и цели

Фреймворк предназначен для построения SPA-порталов без классического backend-рендера, с акцентом на:

- Web Components + Shadow DOM
- Жёсткую инкапсуляцию модулей
- Ленивую загрузку секций при прокрутке
- Возможность внедрения внешних сервисов (API, микросервисы)
- Минимальную зависимость от сборщиков и Node.js

Портал может обслуживаться обычным **nginx** как статический сайт.

2. Общая архитектура



3. Базовый фундамент: ShadowModule

Все компоненты портала наследуются от одного базового класса. Он обеспечивает:

- Shadow DOM
- Изоляцию стилей
- Общую точку расширения

```
// shadow-module.js
export class ShadowModule extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
  }

  connectedCallback() {
    // hook для наследников
  }
}
```

```
customElements.define('shadow-module', ShadowModule);
```

4. LazyModule — ленивый фундамент

LazyModule расширяет ShadowModule и добавляет механизм ленивой активации через IntersectionObserver.

```
// lazy-module.js
import { ShadowModule } from './shadow-module.js';

export class LazyModule extends ShadowModule {
  connectedCallback() {
    this.observer = new IntersectionObserver(entries => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          this.onVisible();
          this.observer.disconnect();
        }
      });
    });
  }
}
```

```
        }
    });
});
this.observer.observe(this);
}

onVisible() {
    // реализуется в наследниках
}
}
```

5. ArticleModule — оркестратор раздела

ArticleModule не знает, что именно он рендерит. Он лишь служит контейнером для секций.

```
// article-module.js
import { ShadowModule } from './shadow-module.js';
```

```
export class ArticleModule extends ShadowModule {  
  constructor() {  
    super();  
    this.shadowRoot.innerHTML = ``;  
  }  
}  
  
customElements.define('article-module', ArticleModule);
```

6. ArticleSection — MD-секция

ArticleSection:

- наследует LazyModule
- загружает markdown только при появлении в viewport
- растягивается на 1–2 экрана

```
// article-section.js (схема)
```

```
class ArticleSection extends LazyModule {  
  onVisible() {  
    fetch(this.getAttribute('src'))  
      .then(r => r.text())  
      .then(md => {  
        this.shadowRoot.innerHTML = renderMarkdown(md);  
      });  
  }  
}
```

7. Интерактивные сервисы (пример: QuizSection)

Интерактивные модули:

- также являются ShadowModules
- могут использовать внешние API
- стилизованы и изолированы

```
[ quiz-section ]  
  |  
  v  
[ External API ]  
  |  
  v  
[ Rendered UI inside Shadow DOM ]
```

Такие модули могут:

- быть вставлены между MD-секциями
- работать автономно
- переиспользоваться в других разделах

8. PortalShell — SPA-оболочка

PortalShell отвечает за:

- роутинг (history API)
- меню

- сборку раздела из секций

```
// app.js (логика)
switch (location.pathname) {
  case '/quiz':
    sections = [
      '/content/articles/quiz/a.md',
      'quiz',
      '/content/articles/quiz/d.md'
    ];
  }
}
```

Важно: секции создаются через `createElement`, а не `innerHTML`, чтобы сохранить Shadow DOM.

9. Принципы расширения

- Любой новый тип секции = новый Web Component
- Никакой жёсткой связи между разделами
- Контент и логика разделены

- Backend может быть добавлен позже (Python, Orthanc API)

10. Что дальше

- JSON-описания разделов
- Серверный API для динамических данных
- Авторизация
- Интеграция с PACS / Orthanc
- Каталог микросервисных модулей

Документ предназначен для технических специалистов. Контентные файлы (.md) намеренно исключены из описания.